

Трансляция презентации (во время очных лекций)

<https://clck.ru/3D3Efj>



При просмотре презентации в PDF для отображения анимаций на слайдах необходимо использовать Acrobat Reader, KDE Okular, PDF-XChange, Foxit Reader, браузер Firefox. Для браузеров на движке Chrome (Edge, Яндекс, Опера, ...) необходимо использовать [PDF.js](#) с опцией «Enable active content (JavaScript) in PDFs».



Промышленное программирование

Тема 4

Введение в контейнеризацию

Гаврилов Андрей Геннадьевич

кандидат технических наук, доцент

Кафедра Информационных технологий и вычислительных систем
МГТУ «СТАНКИН»

Обновлено: 25 мая 2026 г.

- 1 Задача контейнеризации
- 2 Docker. Работа с контейнерами
- 3 Docker. Создание собственных образов

Раздел 1

Задача контейнеризации

Проблемы традиционной разработки

«А у меня всё работает!» ~_(ツ)_/~, «Machine-Driven Development»

Конфигурация оборудования и программного обеспечения на компьютере разработчика и пользователя почти гарантированного будет различаться.

Мемчики

«Matrix from Hell», «Dependency Hell», «Dependency swamp», «DLL Hell»

Программа требует библиотеки, которые требуют библиотеки, которые требуют ...
В сложных проектах часто возникают конфликты библиотек и зависимостей.

Пример из жизни

«Snowflake-server»

Многие готовые решения крайне сложны в развертывании и настройке, особенно новичкам.

Пример — сервер разворачивается по мануалу из 87 шагов, к которому боятся прикоснуться, потому что «он держится на соплях и заклинаниях».

Проблемы традиционной разработки

«А у меня всё работает!» ~_(\ツ)_/, «Machine-Driven Development»

Конфигурация оборудования и программного обеспечения на компьютере разработчика и пользователя почти гарантированного будет различаться.

Мемчики

«Matrix from Hell», «Dependency Hell», «Dependency swamp», «DLL Hell»

Программа требует библиотеки, которые требуют библиотеки, которые требуют ...
В сложных проектах часто возникают конфликты библиотек и зависимостей.

Пример из жизни

«Snowflake-server»

Многие готовые решения крайне сложны в развертывании и настройке, особенно новичкам.

Пример — сервер разворачивается по мануалу из 87 шагов, к которому боятся прикоснуться, потому что «он держится на соплях и заклинаниях».

Проблемы традиционной разработки

«А у меня всё работает!» ~_(_)/~, «Machine-Driven Development»

Конфигурация оборудования и программного обеспечения на компьютере разработчика и пользователя почти гарантированного будет различаться.

Мемчики

«Matrix from Hell», «Dependency Hell», «Dependency swamp», «DLL Hell»

Программа требует библиотеки, которые требуют библиотеки, которые требуют ... В сложных проектах часто возникают конфликты библиотек и зависимостей.

Пример из жизни

«Snowflake-server»

Многие готовые решения крайне сложны в развертывании и настройке, особенно новичкам.

Пример — сервер разворачивается по мануалу из 87 шагов, к которому боятся прикоснуться, потому что «он держится на соплях и заклинаниях».

Как теперь жить?

Способы изоляции:

- Выделенные сервера
- Гипервизоры
- Контейнеры

Виртуальная машина —

это программная (или логическая) копия физического компьютера. Она эмулирует аппаратное обеспечение и позволяет запускать внутри себя гостевую операционную систему так, как будто та работает на реальном «железе».

Гипервизор —

Гипервизор — это программный слой, который создаёт и запускает виртуальные машины. Он позволяет одной физической компьютерной системе («хосту») разделить свои ресурсы между несколькими изолированными средами.

Контейнер —

это легковесная, изолированная среда для запуска приложений, которая использует ядро хостовой операционной системы, а не эмулирует аппаратное обеспечение.

Как теперь жить?

Способы изоляции:

- Выделенные сервера
- Гипервизоры
- Контейнеры

Виртуальная машина —

это программная (или логическая) копия физического компьютера. Она эмулирует аппаратное обеспечение и позволяет запускать внутри себя гостевую операционную систему так, как будто та работает на реальном «железе».

Гипервизор —

Гипервизор — это программный слой, который создаёт и запускает виртуальные машины. Он позволяет одной физической компьютерной системе («хосту») разделить свои ресурсы между несколькими изолированными средами.

Контейнер —

это легковесная, изолированная среда для запуска приложений, которая использует ядро хостовой операционной системы, а не эмулирует аппаратное обеспечение.

Как теперь жить?

Способы изоляции:

- Выделенные сервера
- Гипервизоры
- Контейнеры

Виртуальная машина —

это программная (или логическая) копия физического компьютера. Она эмулирует аппаратное обеспечение и позволяет запускать внутри себя гостевую операционную систему так, как будто та работает на реальном «железе».

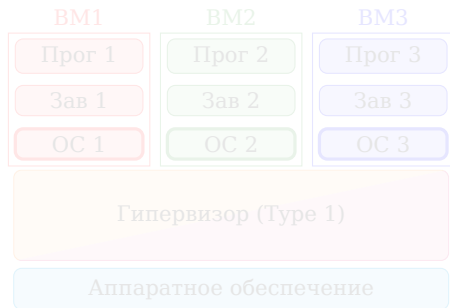
Гипервизор —

Гипервизор — это программный слой, который создаёт и запускает виртуальные машины. Он позволяет одной физической компьютерной системе («хосту») разделить свои ресурсы между несколькими изолированными средами.

Контейнер —

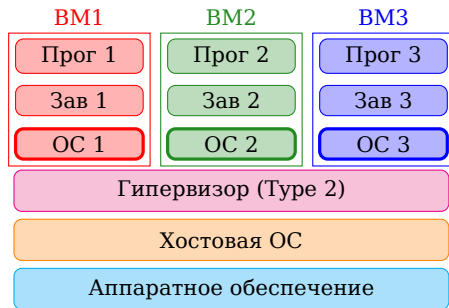
это легковесная, изолированная среда для запуска приложений, которая использует ядро хостовой операционной системы, а не эмулирует аппаратное обеспечение.

Виды гипервизоров



Тип 1. Гипервизоры «голого железа» («bare metal»). Является миниатюрной операционной системой или работает на низком уровне.

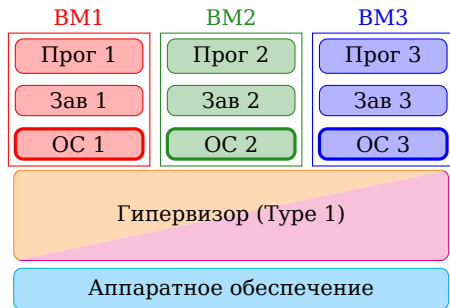
- VMware ESXi
- Microsoft Hyper-V
- KVM (Kernel-based Virtual Machine)
- Xen



Тип 2. Гипервизоры «хостовые». Запускаются как обычное приложение внутри уже установленной операционной системы (хоста).

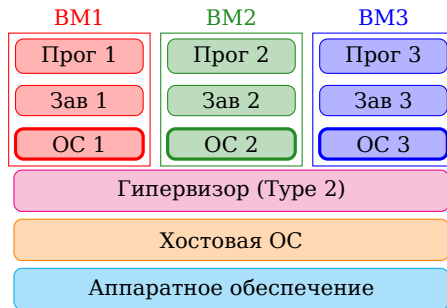
- Oracle VirtualBox
- VMware Workstation
- QEMU (*)
- Parallels Desktop

Виды гипервизоров



Тип 1. Гипервизоры «голого железа» («bare metal»). Является миниатюрной операционной системой или работает на низком уровне.

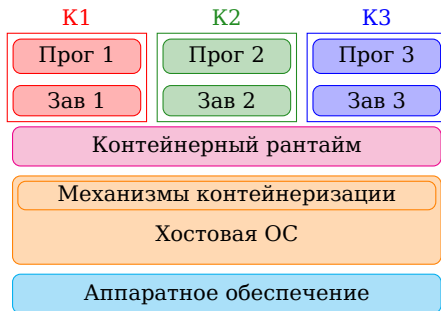
- VMware ESXi
- Microsoft Hyper-V
- KVM (Kernel-based Virtual Machine)
- Xen



Тип 2. Гипервизоры «хостовые». Запускаются как обычное приложение внутри уже установленной операционной системы (хоста).

- Oracle VirtualBox
- VMware Workstation
- QEMU (*)
- Parallels Desktop

Контейнеризация (Linux only!)



Механизмы контейнеризации:

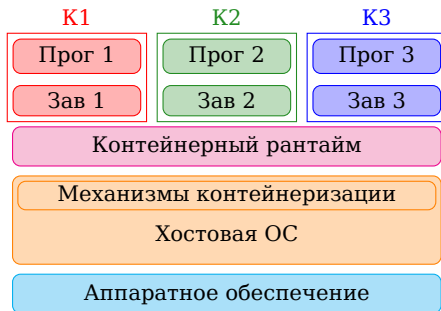
- **Namespaces** — даёт изоляцию (видимость). Создаёт иллюзию, что контейнер — это единственная система
- **Cgroups** (Control groups) — ограничитель ресурсов
- **Chroot / pivot_root** — даёт изоляцию файловой системы.

Контейнерный рантайм (container runtime) —

это ПО, которое отвечает за запуск и остановку контейнеров.

- **Низкоуровневый** (low-level runtime) — работает напрямую с механизмами контейнеризации. (*runc, crun*)
- **Высокоуровневый** (high-level runtime) — работает с образами, сетями, томами, хранилищами. Для запуска контейнера вызывает низкоуровневый рантайм. (*Docker Engine, containerd, CRI-C, Podman*)

Контейнеризация (Linux only!)



Механизмы контейнеризации:

- **Namespaces** — даёт изоляцию (видимость). Создаёт иллюзию, что контейнер — это единственная система
- **Cgroups** (Control groups) — ограничитель ресурсов
- **Chroot / pivot_root** — даёт изоляцию файловой системы.

Контейнерный рантайм (container runtime) —

это ПО, которое отвечает за запуск и остановку контейнеров.

- **Низкоуровневый** (low-level runtime) — работает напрямую с механизмами контейнеризации. (*runc, crun*)
- **Высокоуровневый** (high-level runtime) — работает с образами, сетями, томами, хранилищами. Для запуска контейнера вызывает низкоуровневый рантайм. (*Docker Engine, containerd, CRI-O, Podman*)

- 1 Задача контейнеризации
- 2 Docker. Работа с контейнерами
- 3 Docker. Создание собственных образов

Раздел 2

Docker. Работа с контейнерами



Установка Docker

Инструкции по установке Docker



<https://docs.docker.com/engine/install/>

```
:~$ docker --version  
Docker version 29.4.3, build 055a478
```

Структура команды docker:

docker [контекст] [опции_глобальные]

КОМАНДА [аргументы_команды] [опции_команды]

[ОБРАЗ/КОНТЕЙНЕР/ ...] [команда_в_контейнере] [аргументы_команды_контейнера]

Или более просто:

docker КОМАНДА [ОБРАЗ/КОНТЕЙНЕР/ ...] [команда_в_контейнере]

Примеры:

```
docker run -d --name web -p 8080:80 nginx:alpine
```

```
docker exec -it my_container bash -c "echo Hello && ls -la"
```

Hello from Docker!

```
:~$ sudo docker run hello-world
```

```
Unable to find image 'hello-world:latest' locally
```

```
latest: Pulling from library/hello-world
```

```
4f55086f7dd0: Pull complete
```

```
d5e71e642bf5: Download complete
```

```
Digest: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
```

```
Status: Downloaded newer image for hello-world:latest
```

```
Hello from Docker!
```

```
This message shows that your installation appears to be working correctly.
```

```
To generate this message, Docker took the following steps:
```

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

```
...
```


Основные определения Docker (1/2)

Образ (image) —

неизменяемый шаблон (слепок), содержащий всё необходимое для запуска приложения: операционную систему, код, библиотеки, переменные окружения и настройки. Он только для чтения.

Контейнер (container) —

запущенный экземпляр образа. Это изолированный процесс (или группа процессов) на хосте, который получил свою копию файловой системы из образа, собственную сеть и ограниченные ресурсы. Несколько контейнеров могут использовать один и тот же образ, экономя дисковое пространство и память.

Docker Hub —

это облачный реестр (container registry), предназначенный для хранения, совместного использования и распространения Docker-образов. Это официальный и самый популярный сервис такого рода от компании Docker (но не единственный).

Основные определения Docker (2/2)

Docker Client (CLI) —

Программа, с которой напрямую взаимодействует пользователь. Принимает команды (`docker run`, `docker ps`), преобразует их в HTTP-запросы и отправляет демону.

Docker Daemon (dockerd) —

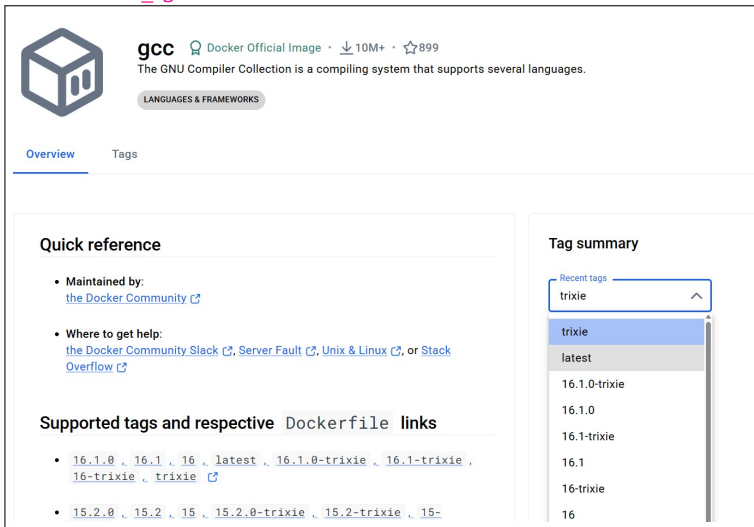
Фоновый процесс (демон), который фактически выполняет команды: запускает контейнеры, управляет образами, сетями, томами.

Том (volume) —

это специальная директория (или область хранения) вне объединённой файловой системы контейнера, которая позволяет сохранять данные даже после удаления контейнера.

Гcc на Docker Hub

https://hub.docker.com/_/gcc



The screenshot shows the Docker Hub page for the `gcc` image. The page header includes the Docker logo, the image name `gcc`, and metadata: "Docker Official Image", "10M+" downloads, and "899" stars. Below this is a description: "The GNU Compiler Collection is a compiling system that supports several languages." and a category tag "LANGUAGES & FRAMEWORKS". The page has two tabs: "Overview" (selected) and "Tags". The "Overview" tab contains a "Quick reference" section with links to the Docker Community, Docker Community Slack, Server Fault, Unix & Linux, and Stack Overflow. It also has a "Supported tags and respective Dockerfile links" section with a list of tags and their corresponding Dockerfile links. On the right side, there is a "Tag summary" section with a dropdown menu showing "Recent tags" and a list of tags: "trixie", "latest", "16.1.0-trixie", "16.1.0", "16.1-trixie", "16.1", "16-trixie", and "16".

gcc Docker Official Image · 10M+ · 899

The GNU Compiler Collection is a compiling system that supports several languages.

LANGUAGES & FRAMEWORKS

Overview Tags

Quick reference

- Maintained by:
[the Docker Community](#)
- Where to get help:
[the Docker Community Slack](#), [Server Fault](#), [Unix & Linux](#), or [Stack Overflow](#)

Supported tags and respective Dockerfile links

- [16.1.0](#), [16.1](#), [16](#), [latest](#), [16.1.0-trixie](#), [16.1-trixie](#), [16-trixie](#), [trixie](#)
- [15.2.0](#), [15.2](#), [15](#), [15.2.0-trixie](#), [15.2-trixie](#), [15-](#)

Tag summary

Recent tags

trixie

latest

16.1.0-trixie

16.1.0

16.1-trixie

16.1

16-trixie

16

Управление образами

```
:~$ sudo docker image pull gcc
Using default tag: latest
latest: Pulling from library/gcc
373364b63187: Pull complete
7df9e11c80c9: Pull complete
...
Digest: sha256:1cfa8769230debf43594ee1b48bc642f9bead8d479f9926bc61d3014bdf3ecc8
Status: Downloaded newer image for gcc:latest
docker.io/library/gcc:latest
```

```
:~$ sudo docker image ls
i Info →   U   In Use
IMAGE              ID                DISK USAGE   CONTENT SIZE  EXTRA
gcc:latest          1cfa8769230d      2.21GB       577MB
hello-world:latest  f9078146db2e      25.9kB       9.49kB       U
```

```
:~$ sudo docker image rm gcc:latest
Untagged: gcc:latest
Deleted: sha256:1cfa8769230debf43594ee1b48bc642f9bead8d479f9926bc61d3014bdf3ecc8
```

```
:~$ sudo docker image ls
i Info →   U   In Use
IMAGE              ID                DISK USAGE   CONTENT SIZE  EXTRA
hello-world:latest  f9078146db2e      25.9kB       9.49kB       U
```

Управление образами

```
:~$ sudo docker image pull gcc
Using default tag: latest
latest: Pulling from library/gcc
373364b63187: Pull complete
7df9e11c80c9: Pull complete
...
Digest: sha256:1cfa8769230debf43594ee1b48bc642f9bead8d479f9926bc61d3014bdf3ecc8
Status: Downloaded newer image for gcc:latest
docker.io/library/gcc:latest

:~$ sudo docker image ls
i Info →   U   In Use
IMAGE              ID                DISK USAGE   CONTENT SIZE  EXTRA
gcc:latest          1cfa8769230d      2.21GB       577MB
hello-world:latest  f9078146db2e      25.9kB       9.49kB       U

:~$ sudo docker image rm gcc:latest
Untagged: gcc:latest
Deleted: sha256:1cfa8769230debf43594ee1b48bc642f9bead8d479f9926bc61d3014bdf3ecc8

:~$ sudo docker image ls
i Info →   U   In Use
IMAGE              ID                DISK USAGE   CONTENT SIZE  EXTRA
hello-world:latest  f9078146db2e      25.9kB       9.49kB       U
```

Команды управления образами

`docker image *` —

группа команд для управления образами

Команда	Описание
<code>ls</code>	Показать список образов
<code>ls -q</code>	Показать только ID образов
<code>pull [ОБРАЗ]</code>	Скачать образ из реестра
<code>push [ОБРАЗ]</code>	Загрузить образ в реестр
<code>build -t [ИМЯ] [ПУТЬ]</code>	Собрать образ из Dockerfile
<code>build -f [ФАЙЛ] -t [ИМЯ] [ПУТЬ]</code>	Собрать образ с указанием Dockerfile
<code>rm [ОБРАЗ]</code>	Удалить образ
<code>tag [ОБРАЗ] [НОВЫЙ_ТЕГ]</code>	Присвоить тег образу
<code>inspect [ОБРАЗ]</code>	Показать подробную информацию об образе
<code>history [ОБРАЗ]</code>	Показать историю слоёв образа
<code>save -o [ФАЙЛ] [ОБРАЗ]</code>	Сохранить образ в tar-архив
<code>load -i [ФАЙЛ]</code>	Загрузить образ из tar-архива
<code>commit [КОНТЕЙНЕР] [НОВЫЙ_ОБРАЗ]</code>	Создать образ из изменённого контейнера
<code>import [ФАЙЛ] [ОБРАЗ]</code>	Импортировать файловую систему в образ
<code>prune</code>	Удалить все неиспользуемые образы
<code>prune -a</code>	Удалить все неиспользуемые образы (включая не тегированные)

Скачаем компиляторы C++

```
~$ sudo docker image pull gcc:16.1
16.1: Pulling from library/gcc
...
Digest: sha256:1cfa8769230debf43594ee1b48bc642f9bead8d479f9926bc61d3014bdf3ecc8
Status: Downloaded newer image for gcc:16.1
docker.io/library/gcc:16.1

~$ sudo docker image pull silkeh/clang:20-buiiseye
Error response from daemon: failed to resolve reference "docker.io/silkeh/clang:20-buiiseye": d

~$ sudo docker image pull silkeh/clang:20-bullseye
20-bullseye: Pulling from silkeh/clang
...
Digest: sha256:7789959b8e51412e66cefb10135bdefa02a015050a3bcde56d097c07a167559b
Status: Downloaded newer image for silkeh/clang:20-bullseye
docker.io/silkeh/clang:20-bullseye

~$ sudo docker image ls
i Info →   U   In Use
IMAGE              ID              DISK USAGE    CONTENT SIZE  EXTRA
gcc:16.1            1cfa8769230d    2.21GB        577MB
hello-world:latest  f9078146db2e    25.9kB        9.49kB        U
silkeh/clang:20-bullseye 7789959b8e51    1.8GB         395MB
```

Используем образы

```
:~$ cat prog.cpp

#include <iostream>
int main()
{
    std::cout << "Hello!" <<std::endl;
    return 0;
}

:~$ sudo docker run --rm -v ./tmp -w /tmp gcc:16.1 g++ -o prog prog.cpp
:~$ ./prog
Hello!

:~$ sudo docker run --rm -v ./tmp -w /tmp 7789959b8e51 g++ -o prog prog.cpp
:~$ ./prog
Hello!
```


docker run

Ключ	Описание	Пример docker run ...
-it	Интерактивный режим (обычно с bash/sh)	<code>-it ubuntu bash</code>
-d	Запуск контейнера в фоне (detach)	<code>-d nginx</code>
--name	Присвоить контейнеру понятное имя	<code>--name my_db mysql</code>
-p	Пробросить порт хоста в порт контейнера	<code>-p 8080:80 nginx</code>
-v, --volume	Монтировать том или папку хоста	<code>-v /my/data:/app/data ubuntu</code>
--rm	Автоматически удалять контейнер после остановки	<code>--rm alpine echo "hi"</code>
-e, --env	Передать переменную окружения	<code>-e DB_PASS=123 postgres</code>
--restart	Задать политику перезапуска	<code>--restart always redis</code>
-m, --memory	Ограничить максимальное использование RAM	<code>-m 512m node_app</code>
-w, --workdir	Установить рабочую директорию внутри контейнера	<code>-w /app node npm start</code>
--cpus	Ограничить использование CPU (в ядрах)	<code>--cpus=1.5 my_app</code>
--user, -u	Запустить процесс от указанного UID/GID	<code>-u 1000:1000 ubuntu</code>
--read-only	Сделать файловую систему контейнера только для чтения	<code>--read-only nginx</code>

Проникнем в контейнер (1/2)

Терминал 1

```
:~$ sudo docker run -it --rm -v ./tmp -w /tmp gcc:16.1 bash  
  
root@eddd12686841:/tmp# ls  
prog  prog.cpp  
  
root@eddd12686841:/tmp# pwd  
/tmp
```

Терминал 2

```
:~$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
eddd12686841	gcc:16.1	"bash"	39 seconds ago	Up 38 seconds	

Терминал 1

```
root@eddd12686841:/tmp# touch file1
```

Терминал 2

```
:~$ ls  
file1  prog  prog.cpp
```

Проникнем в контейнер (1/2)

Терминал 1

```
:~$ sudo docker run -it --rm -v ./tmp -w /tmp gcc:16.1 bash

root@eddd12686841:/tmp# ls
prog  prog.cpp

root@eddd12686841:/tmp# pwd
/tmp
```

Терминал 2

```
:~$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
eddd12686841	gcc:16.1	"bash"	39 seconds ago	Up 38 seconds	

Терминал 1

```
root@eddd12686841:/tmp# touch file1
```

Терминал 2

```
:~$ ls
file1  prog  prog.cpp
```

Проникнем в контейнер (2/2)

Терминал 1

```
root@eddd12686841:/tmp# g++ -o prog2 prog.cpp

root@eddd12686841:/tmp# ./prog2
Hello!

root@eddd12686841:/tmp# exit
exit
```

Терминал 2

```
:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS        NAMES

:~$ ./prog2
Hello!
```

Фоновый запуск контейнера

```
:~$ sudo docker run -d --rm -p 8000:80 nginx
```

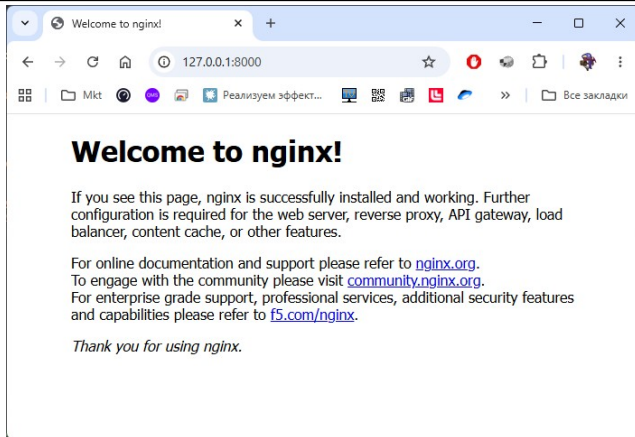
```
...
```

```
Status: Downloaded newer image for nginx:latest
```

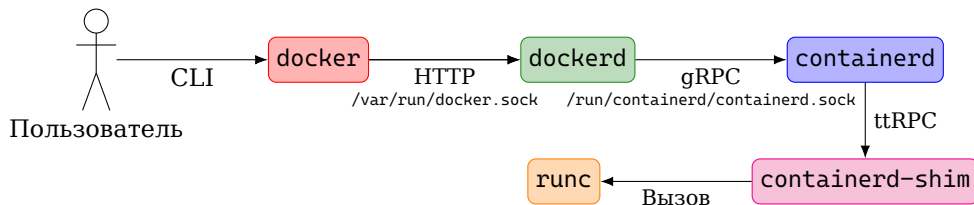
```
299be21624064dfae163281b2450570fc20537624ac73fb1265bccae25682282
```

```
L:~$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
299be2162406	nginx	"/docker-entrypoint...."	12 seconds ago	Up 12 seconds	0.0.0.0:8000

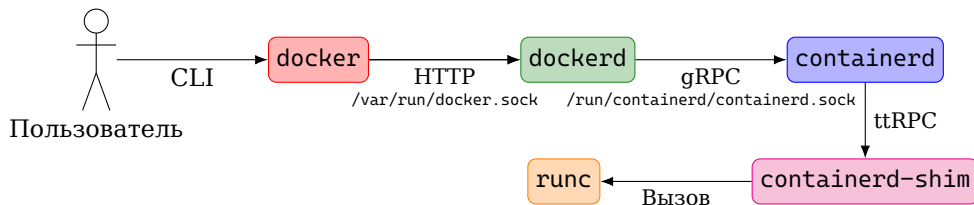


Docker, зачем так сложно?



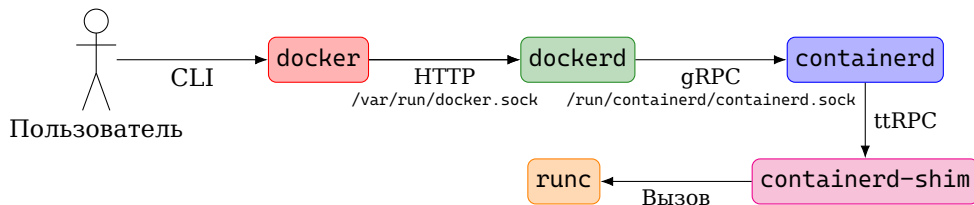
- **docker (CLI)** — интерпретирует команды пользователя и отправляет их через `/var/run/docker.sock` демону **dockerd**.
- **dockerd** — управляет контейнерами, сетями, томами и образами, взаимодействуя с **containerd** по протоколу **gRPC**.
- **containerd** — высокоуровневый рантайм; принимает задачи от **dockerd** и запускает **containerd-shim**.
- **containerd-shim** — процесс-посредник для одного контейнера; переживает перезапуск **containerd**.
- **runc** — низкоуровневый рантайм; через `fork` / `exec` создаёт Namespaces, Cgroups и запускает процесс контейнера, после чего завершается, оставляя **shim** родителем.

Docker, зачем так сложно?



- `docker` (CLI) — интерпретирует команды пользователя и отправляет их через `/var/run/docker.sock` демону `dockerd`.
- `dockerd` — управляет контейнерами, сетями, томами и образами, взаимодействуя с `containerd` по протоколу gRPC.
- `containerd` — высокоуровневый рантайм; принимает задачи от `dockerd` и запускает `containerd-shim`.
- `containerd-shim` — процесс-посредник для одного контейнера; переживает перезапуск `containerd`.
- `runc` — низкоуровневый рантайм; через `fork` / `exec` создаёт Namespaces, Cgroups и запускает процесс контейнера, после чего завершается, оставляя `shim` родителем.

Docker, зачем так сложно?



- `docker` (CLI) — интерпретирует команды пользователя и отправляет их через `/var/run/docker.sock` демону `dockerd`.
- `dockerd` — управляет контейнерами, сетями, томами и образами, взаимодействуя с `containerd` по протоколу gRPC.
- `containerd` — высокоуровневый рантайм; принимает задачи от `dockerd` и запускает `containerd-shim`.
- `containerd-shim` — процесс-посредник для одного контейнера; переживает перезапуск `containerd`.
- `runc` — низкоуровневый рантайм; через `fork` / `exec` создаёт Namespaces, Cgroups и запускает процесс контейнера, после чего завершается, оставляя `shim` родителем.

- 1 Задача контейнеризации
- 2 Docker. Работа с контейнерами
- 3 Docker. Создание собственных образов

Раздел 3

Docker. Создание собственных образов

Dockerfile

docker build —

это команда, которая читает инструкции из Dockerfile и создаёт (собирает) Docker-образ. В процессе сборки выполняются команды из Dockerfile (RUN, COPY, ADD и т.д.), а результат сохраняется как образ, готовый к запуску контейнеров.

Dockerfile —

это текстовый файл (без расширения), содержащий последовательность инструкций для автоматической сборки Docker-образа.

dockerfile

```
1 FROM ubuntu:22.04
2 COPY ./app /app
3 RUN make /app
4 CMD python /app/main.py
```

Основные инструкции dockerfile

Ин-ция	Описание	Пример
FROM	Задаёт базовый образ	FROM ubuntu:22.04
RUN	Выполняет команду во время сборки	RUN apt update && apt install -y python3
COPY	Копирует файлы с хоста в образ	COPY . /app
WORKDIR	Устанавливает рабочую директорию	WORKDIR /app
CMD	Команда по умолчанию при запуске контейнера (может быть переопределена)	CMD ["python "app.py"]
ENTRYPOINT	Основная команда контейнера (не может быть переопределена полностью)	ENTRYPOINT ["python"]
ENV	Устанавливает переменную окружения (сохраняется в образе)	ENV NODE_ENV=production
EXPOSE	Документирует, какой порт использует приложение	EXPOSE 8080
VOLUME	Создаёт точку монтирования для тома	VOLUME /data

Дополнительные инструкции dockerfile

Ин-ция	Описание	Пример
ADD	Копирует файлы, распаковывает tar или скачивает по URL	ADD https://example.com/file.tar.gz /
ARG	Переменная только для сборки (не сохраняется в образе)	ARG VERSION=1.0
USER	Задаёт пользователя для выполнения команд	USER appuser
LABEL	Добавляет метаданные в формате ключ=значение	LABEL version="1.0"
HEALTHCHECK	Определяет команду для проверки состояния контейнера	HEALTHCHECK CMD curl -f http://localhost/
SHELL	Меняет командную оболочку по умолчанию	SHELL ["/bin/bash c"]
STOPSIGNAL	Задаёт системный сигнал для остановки контейнера	STOPSIGNAL SIGTERM
ONBUILD	Инструкция, которая выполняется при использовании образа как базового	ONBUILD RUN pip install -r requirements.txt

Упакуем программу в образ

prog_sleep.cpp

```
1 #include <unistd.h>
2 #include <iostream>
3
4 int main()
5 {
6     while (1)
7     {
8         std::cout << "Hello" << std::endl;
9         sleep(1);
10    }
11    return 0;
12 }
```

dockerfile

```
1 FROM debian:latest
2 COPY prog_sleep /usr/local/bin/prog_sleep
3 RUN chmod +x /usr/local/bin/prog_sleep
4 ENTRYPOINT ["/usr/local/bin/prog_sleep"]
```

Соберём образ

```
:~$ sudo docker run --rm -v ./tmp -w /tmp gcc:16.1 g++ prog_sleep.cpp -o prog_sleep
:~$ sudo docker build -t sleep_app .
[+] Building 10.8s (8/8) FINISHED
...
⇒ [1/3] FROM docker.io/library/debian:latest sha256:4ae67669760b807c19f23902a3fd7c121a6a70cf2a
...
⇒ [2/3] COPY prog_sleep /usr/local/bin/prog_sleep
⇒ [3/3] RUN chmod +x /usr/local/bin/prog_sleep
⇒ exporting to image
...
⇒ ⇒ unpacking to docker.io/library/sleep_app:latest

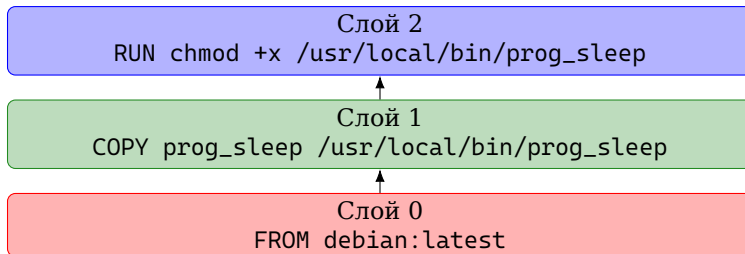
:~$ sudo docker image ls
i Info →   U In Use
IMAGE          ID          DISK USAGE  CONTENT SIZE  EXTRA
...
sleep_app:latest  f5cd4b334690    183MB        49.3MB

:~$ sudo docker run --rm sleep_app
Hello
Hello
Hello
Hello
Hello
```

Слой

Слой (layer) —

это результат выполнения одной инструкции в Dockerfile, которая изменяет файловую систему. Каждый слой содержит только изменения (разницу) относительно предыдущего слоя.



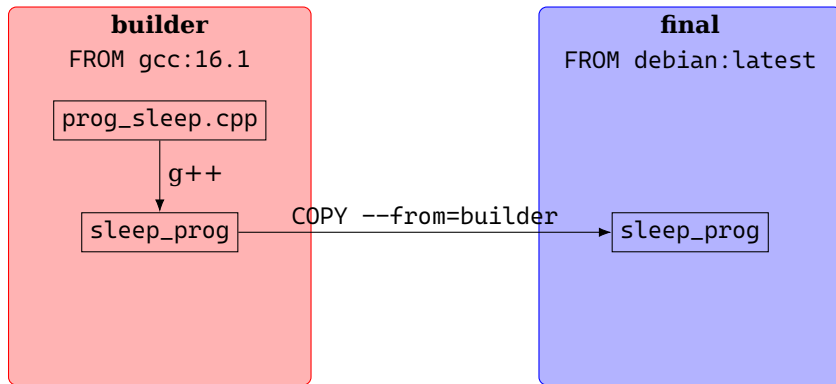
```
:~$ sudo docker image history sleep_app
```

IMAGE	CREATED	CREATED BY	SIZE	COMME
f5cd4b334690	44 minutes ago	ENTRYPOINT ["/usr/local/bin/prog_sleep"]	0B	build
<missing>	44 minutes ago	RUN /bin/sh -c chmod +x /usr/local/bin/prog...	4.1kB	builc
<missing>	44 minutes ago	COPY prog_sleep /usr/local/bin/prog_sleep # ...	32.8kB	builc
<missing>	6 days ago	# debian.sh --arch 'amd64' out/ 'trixie' '1...	134MB	debuer

Многоэтапная сборка

Multistage сборка (многоэтапная сборка) —

это техника в Docker, использующая несколько инструкций FROM в одном Dockerfile для разделения процесса сборки и финального образа. Первые этапы (builder) содержат инструменты сборки (компиляторы, зависимости), последний этап (final) копирует только готовые артефакты.



Обновим dockerfile для multistage

dockerfile

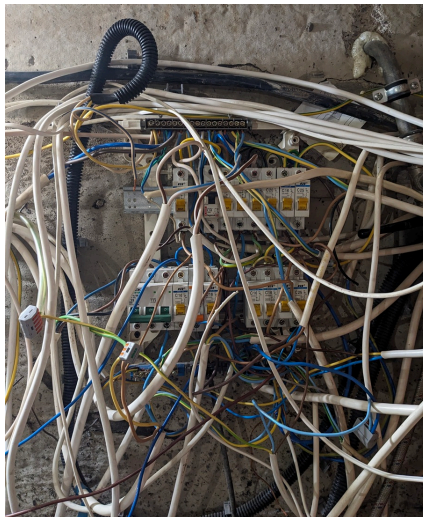
```
1 FROM gcc:16.1 AS builder
2 WORKDIR /tmp
3 COPY prog_sleep.cpp .
4 RUN g++ prog_sleep.cpp -o prog_sleep
5
6 FROM debian:latest
7 COPY --from=builder /tmp/prog_sleep /usr/local/bin/prog_sleep
8 RUN chmod +x /usr/local/bin/prog_sleep
9 ENTRYPOINT ["/usr/local/bin/prog_sleep"]
```

```
:~$ sudo docker build -t sleep_app .
[+] Building 2.2s (13/13) FINISHED
```

```
:~$ sudo docker run sleep_app
Hello
Hello
Hello
Hello
Hello
```

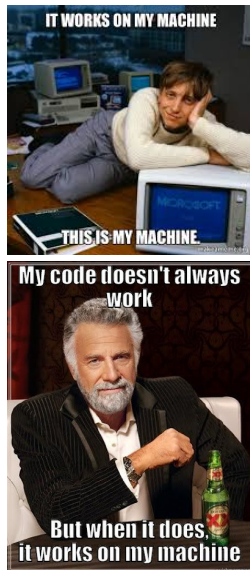
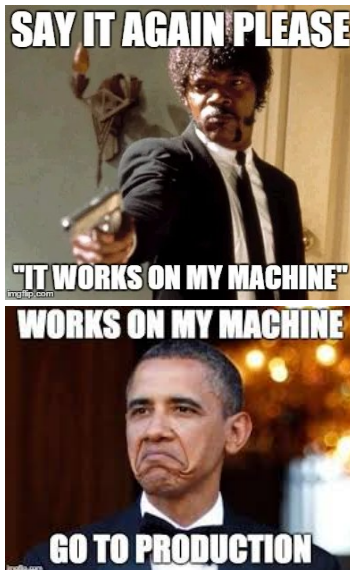
Приложения

Не надо так делать



К лекции

It's works on my machine



К лекции